

Computer Architecture Lab 04 Report

张兆飞

2026-04-03

实验 4 RISC-V 汇编程序编程练习

1. 实验目的

通过编写和运行 RISC-V 汇编程序, 熟悉 RISC-V 基本指令集中的指令, RISC-V 汇编的编程规范, 了解递归函数的调用过程.

2. 实验步骤

2.1. 使用 RV64I 指令集编写 32 位整数乘法函数 mymul

使用循环, 通过不断移位和加法来实现乘法运算. 该函数接受两个参数, 分别存储在 a0 和 a1 寄存器中, 返回结果存储在 a0 寄存器中.

```
1      .text
2      .globl mymul
3      .type mymul,@function
4 mymul:
5      addi    sp,sp,-32
6      sd      ra,24(sp)
7      sd      s0,16(sp)
8      addi    s0,sp,32
9
10     # Your code here
11     # Begin my code
12     mv       a2,x0          # Initialize a2
13
14 mymul_loop:
15     beqz     a1,mymul_end    # Finished if a1 is 0
16     andi     t0,a1,1         # LSB of a1
17     beqz     t0,mymul_skip   # Skip the addition if 0
18
19 mymul_add:
20     addw     a2,a2,a0        # Add to the result
21
22 mymul_skip:
23     slliw    a0,a0,1         # Multiply by 2
24     srliw    a1,a1,1         # Next bit
25     j        mymul_loop     # Jump back
26
27 mymul_end:
28     mv       a0,a2          # Return value
29     # End my code
30
31     ld       ra,24(sp)
32     ld       s0,16(sp)
33     addi     sp,sp,32
34     ret
35
```

图 1: Source Code of mymul

2.2. 使用 RV64I 指令集和 mymul 函数实现自然数阶乘函数 myfac

使用递归的方式实现阶乘函数, 输入为 0 时为 base case. 当输入大于 0 时, 通过递归调用 myfac 来计算阶乘, 并使用 mymul 来计算当前数与递归结果的乘积.

```
1      .text
2      .globl myfac
3      .type myfac,@function
4 myfac:
5      addi    sp,sp,-32
6      sd      ra,24(sp)
7      sd      s0,16(sp)
8      addi    s0,sp,32
9
10     # Your code here
11     # Begin my code
12     beqz     a0,myfac_base    # Base case
13     mv      s0,a0            # Store a0 to s0
14
15 myfac_recur:
16     addi     a0,a0,-1
17     jal      ra,myfac
18     mv      a1,s0
19     jal      ra,mymul
20     j        myfac_end
21
22 myfac_base:
23     li       a0,1            # 0! = 1
24
25 myfac_end:
26     # End my code
27
28     ld      ra,24(sp)
29     ld      s0,16(sp)
30     addi    sp,sp,32
31     ret
32
```

图 2: Source Code of myfac

2.3. 编译和测试结果

编译后直接运行, 与预期结果完全一致.



```
jz2024@JZ2024:~
jz2026-ubuntu-ca@JZ2026-Ubuntu-CA:exercise$ make
riscv64-unknown-elf-gcc -g -c mul.s fac.s main.c
riscv64-unknown-elf-gcc -g mul.o fac.o main.o -o test
jz2026-ubuntu-ca@JZ2026-Ubuntu-CA:exercise$ qemu-riscv64 test
Test for mymul:
0*5=0
(-3)*0=0
3*5=15
(-3)*5=-15
3*(-5)=-15
(-3)*(-5)=15

Test for myfac:
0!=1
1!=1
2!=2
3!=6
4!=24
5!=120
6!=720
7!=5040
8!=40320
9!=362880
jz2026-ubuntu-ca@JZ2026-Ubuntu-CA:exercise$
```

图 3: Output of mymul and myfac

3. 实验分析与总结

通过本次实验, 熟悉了 RISC-V 汇编语言的基本语法和指令使用, 了解了递归函数的调用过程以及如何在汇编中实现递归. 在编写 `mymul` 函数时, 通过不断移位和加法来实现乘法运算, 加深了对位运算的理解. 在实现 `myfac` 函数时, 通过递归调用来计算阶乘, 加深了对递归算法的理解. 实验结果与预期完全一致, 验证了代码的正确性.

4. 实验收获与建议

本次实验锻炼了使用 RISC-V 汇编语言进行编程的能力, 加深了对 RISC-V 指令集和计算机体系结构的理解, 这对于软件开发和硬件设计都有重要的意义. 在编写汇编代码时, 熟悉了寄存器的使用和函数调用约定, 有利于确保代码的正确性和可维护性.

源代码中 `mymul` 作为 leaf function, 并不需要保存 `ra` 寄存器, 也不一定使用 `s0` 寄存器, 因此可以去除其代码中相关的保存和恢复指令. 此外, 也可以考虑在源代码中采用空格而非制表符进行缩进, 以确保在不同编辑器中保持一致的格式, 也更符合现代编程实践.